
ActionProof: Verifying Multi-Step Coding-Agent Transactions

Adarsh Agrawal
Senior Member, IEEE

Abstract

Coding-agent actions are usually checked one at a time, even when the agent is carrying out a transaction whose steps depend on one another. This misses cross-step failures such as stale state, cumulative scope growth, and rollback chains that cannot restore the starting workspace. We present ACTIONPROOF, a transaction-wide checker that links executable action contracts to observed workspace states and verifies the chain before committing it. In 90 controlled cross-step tests, a stateful transaction baseline detects 71 faults, adding cumulative scope reaches 82, and adding a restoration oracle reaches 85. ACTIONPROOF detects all 90 with no false blocks on 90 valid controls. On eight acquired SWE-bench workflows, it detects 24/24 cross-step faults compared with 19/24 for the stateful baseline. We also study 21 SWE-agent trajectories from 14 distinct instances. Contracts generated from issue and repository context reach 0.714 instance-level needed-write preservation, compared with 0.500 for transaction authorization and 1.000 for trajectory-derived reference contracts. The result is a contract-relative guarantee: once a useful executable contract exists, transaction-wide checking can catch failures that per-action authorization misses.

1 Introduction

Coding agents rarely make only one change. A repair may inspect a repository, edit source, add a regression test, update configuration, and run a release check. Yet authorization is usually applied to each tool call in isolation. Every action can look acceptable on its own while the sequence consumes stale state, exceeds a write budget, or loses the information needed for rollback.

Consider a four-action repair. The first edit is authorized for file a , the second for file b , and both have locally valid rollback descriptions. A per-action monitor accepts both. The transaction can still be invalid if the second action was proved against a state that the first action never produced, if the union of modified paths exceeds the transaction budget, or if the second action destroys the checkpoint required to undo the first. These are composition faults, not failures of individual action syntax.

ACTIONPROOF checks the transaction as a whole. Each action carries a contract describing the state it expects, the state it may produce, the paths and capabilities it may use, an executable postcondition, and a rollback operation. A small checker links accepted steps, tracks the transaction’s total scope, and verifies rollback against the state that was actually observed.

The guarantee is intentionally contract-relative. ACTIONPROOF can establish that an execution follows a supplied contract; it cannot recover intent that the contract never expressed. We therefore evaluate both sides of the problem: checking complete contracts and acquiring useful contracts from repository context.

This paper makes three contributions:

1. A transaction model that checks state continuity, cumulative scope, observed execution, receipt integrity, and rollback restoration.

2. An evaluation that adds one verification component at a time, showing what each component contributes on controlled and acquired workflows.
3. A natural-trace and contract-acquisition study that identifies executable postconditions and rollback descriptions as the main remaining acquisition challenge.

2 ActionProof

2.1 Setting and threat model

Let a workspace state $x \in \mathcal{X}$ be a finite map from normalized relative paths to byte strings and metadata. An action a is a deterministic operation in the supported interpreter, such as a bounded file write, local command, archive construction, or materialization into a disposable release directory. The interpreter rejects path escapes, unsupported operations, malformed content, and writes outside the declared scope.

The agent and contract generator are untrusted. They may provide incorrect fields, reuse an old receipt, grow the transaction scope one step at a time, or act on state that changed after authorization. The trusted side consists of the fixed policy, interpreter, state snapshotter, contract checker, solver interface, execution monitor, and rollback checker. The guarantee therefore depends on faithful snapshots, correct policy and contract interpretation, complete mediation of supported actions, and correct rollback execution. Z3 and cvc5 provide solver implementation diversity, but both consume the same SMT-LIB encoding and therefore do not protect against a shared encoding error.

A well-formed contract can still describe the wrong action. ACTIONPROOF protects the boundary between contract and execution, not the boundary between a user's unstated intent and the contract. This motivates the acquisition study in Section 3.5.

2.2 Contracts and ledger state

Definition 1 (Action contract). *A contract for step i is*

$$c_i = (\iota_i, a_i, \phi_i, \psi_i, S_i, K_i, \tau_i, \rho_i),$$

where ι_i is an intent identifier, a_i is the normalized action descriptor, $\phi_i : \mathcal{X} \rightarrow \{0, 1\}$ is the pre-state predicate, $\psi_i : \mathcal{X} \rightarrow \{0, 1\}$ is the post-state predicate, $S_i \subseteq \mathcal{P}$ is the authorized scope, K_i is the required capability set, τ_i is an executable postcondition, and ρ_i is a pre-authorized rollback description.

Definition 2 (Composition ledger). *After step i , the ledger state is*

$$q_i = (h_i, W_i, R_i),$$

where $h_i \in \{0, 1\}^{256}$ is the receipt-chain head, $W_i = \bigcup_{j=1}^i S_j$ is cumulative scope, and $R_i = [\rho_1, \dots, \rho_i]$ is the rollback stack. The genesis state is $q_0 = (h_0, \emptyset, [])$.

2.3 Two-phase transition

Given observed workspace x_{i-1} , ledger q_{i-1} , contract c_i , and fixed policy P , the checker first verifies:

$$\phi_i(x_{i-1}) = 1, \tag{1}$$

$$S_i \subseteq P.\text{allowed_scope}, \tag{2}$$

$$W_{i-1} \cup S_i \subseteq P.\text{transaction_budget}, \tag{3}$$

$$K_i \subseteq P.\text{capabilities}. \tag{4}$$

It also checks that the action, policy, contract, and active snapshot match their signed digests. The invariants are encoded as an SMT violation query. Authorization requires both Z3 and cvc5 to return UNSAT. They consume the same SMT-LIB encoding, so agreement provides implementation diversity but does not protect against a shared encoding error.

After pre-execution authorization, the monitor executes a_i to obtain x_i . Commit requires

$$\psi_i(x_i) = 1 \quad \text{and} \quad \tau_i(x_i) = 0.$$

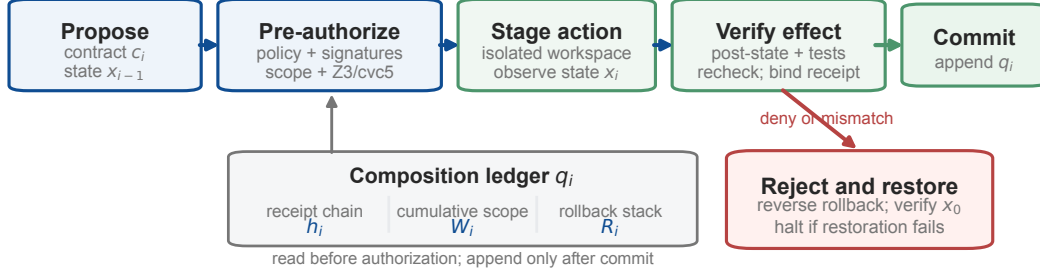


Figure 1: ACTIONPROOF’s fail-closed transaction workflow. The checker separates pre-authorization from staged execution, binds receipts to observed effects, and advances the composition ledger only after the signed contract is satisfied. Any failure enters verified reverse-order rollback.

The monitor then extends the ledger:

$$h_i = H(h_{i-1} \parallel \text{canonical}(c_i) \parallel \text{snapshot}(x_i)), \quad (5)$$

$$W_i = W_{i-1} \cup S_i, \quad (6)$$

$$R_i = R_{i-1} \parallel [\rho_i]. \quad (7)$$

Parsing, policy, state, solver, execution, and postcondition failures all block the commit.

2.4 Failure and rollback

If step k fails after earlier steps have committed, the monitor applies $\rho_{k-1}, \dots, \rho_1$ in reverse order. After each rollback operation it checks the corresponding signed pre-state predicate. A transaction is marked ROLLED-BACK only if the final workspace equals the initial signed state. Otherwise the ledger enters INCOMPLETE-ROLLBACK, halts further agent actions, preserves the partial trace, and requests operator handling. Rollback descriptions are pre-authorized by the original contracts; successful restoration is still checked against observed state.

2.5 Bounded composition guarantee

Theorem 1 (Contract-relative composition soundness). *Assume a trusted fail-closed monitor, interpreter, policy, signature verifier, SMT encoder, hash function, and snapshot mechanism. For every accepted prefix $(c_1, \dots, c_n)$ producing observed states $(x_0, \dots, x_n)$, the following properties hold:*

1. $\phi_i(x_{i-1}) = 1$ for every accepted step i ;
2. $\bigcup_{i=1}^n S_i \subseteq P.\text{transaction_budget}$;
3. $\psi_i(x_i) = 1$ and $\tau_i(x_i) = 0$ for every committed step;
4. every receipt binds the prior head, canonical contract, and observed post-state snapshot; and
5. if a failure occurs and the signed rollback descriptions are correct and executable, reverse-order rollback restores x_0 .

Proof sketch. The base state satisfies the empty-scope and genesis-receipt conditions. Equations 1–4 establish the induction step before execution. Post-state and postcondition checks establish execution conformance before the ledger advances. The receipt recurrence preserves chain integrity, and set union preserves the cumulative-scope invariant. Reverse-order rollback restores x_0 by repeated application of the signed restoration descriptions and observed equality checks. The final clause is conditional on the rollback descriptions themselves being correct. The theorem does not establish the semantic correctness of the supplied intent or contract.

3 Evaluation

3.1 Questions

We evaluate five questions:

1. Do composition checks detect cross-step faults missed by per-action authorization?
2. Which mechanism components account for the difference?
3. Does the result transfer to contracts acquired from repository workflows?
4. How does contract source affect authorization on natural agent traces?
5. What operational costs and failure boundaries appear in supporting execution studies?

3.2 Controlled composition frame

The controlled evaluation contains 30 four-action workflows over disposable workspaces. Each workflow has one valid chain and three cross-step variants:

- **Continuity:** a step is proved against a pre-state not produced by the preceding action.
- **Cumulative scope:** every action is locally in scope, but the union of modified paths exceeds the transaction budget.
- **Rollback:** every local rollback appears executable, but the composed rollback cannot restore the initial state.

This yields 90 faulted transactions and 90 matched valid controls. These tests isolate the properties checked by the transaction model; they do not estimate how often such faults occur in deployed agents.

3.3 Matched baselines

The comparison adds one mechanism at a time:

1. a stateless policy check and a stateless SMT check;
2. stateful transactional verification with step history;
3. stateful verification plus cumulative scope;
4. state, scope, and an executable restoration oracle;
5. ACTIONPROOF with one solver; and
6. full ACTIONPROOF with Z3 and cvc5.

Every method receives the same workflow, contract, policy, and observed action. This comparison isolates mechanism scope rather than model quality. The first two levels approximate the per-action checks common in coding-agent sandboxes and tool monitors; the stateful level adds transaction history, as in history-sensitive policy proposals. The remaining levels are architectural ablations that add cumulative scope, executable restoration, and predicate-verified rollback. We do not claim that each row is a complete reimplement of a particular external system.

3.4 Acquired workflows

We apply the same checks to eight acquired SWE-bench workflows. The extension contains 32 field-drop variants and 24 cross-step variants: eight continuity, eight cumulative-scope, and eight rollback faults. A broader workflow run contains 70 complete workflows across 11 repositories and 210 incomplete variants formed by dropping the final action, dropping a middle action, or reordering actions.

3.5 Natural trajectories and endpoint

The natural-trace corpus contains 21 SWE-agent trajectories drawn from 14 distinct SWE-bench Verified instances across seven repositories [1, 2]. Seven trajectories repeat one high-activity instance under different agent configurations. Every trajectory ends with a submitted patch. Fourteen Terraform and OPA modules are used separately to validate parsing and native tool integration; they are not counted as SWE-agent authorization outcomes.

Instances were selected before authorization experiments using three criteria: the trajectory contained at least two file-writing actions, repository tests could be run from the workspace root, and the submitted patch touched at most 12 files. Of 41 candidate instances with submitted multi-action trajectories, 19 had a usable local test harness and 14 met all three criteria. The seven repeated trajectories were retained because they exercised different agent configurations on the same issue. Repository tests serve as executable postconditions; they are not the official SWE-bench FAIL_TO_PASS task oracle.

The primary endpoint is needed-write preservation:

$$\text{NWP} = 1 \iff C \subseteq A \wedge A \setminus C = \emptyset,$$

where C is the needed write set derived from the submitted trajectory and A is the set of writes retained by the authorization method. NWP measures needed-write retention with no extraneous write. It does not measure task correctness.

The core comparison includes trajectory-derived reference contracts, model-generated contracts built from issue and repository context, and a transactional baseline that commits the complete trajectory write set. Reference contracts use the submitted write set and the available repository tests. The primary unit is the distinct instance ($n = 14$); trajectory-level results ($n = 21$) are secondary. Wilson intervals are reported over distinct-instance outcomes. A sensitivity analysis samples one trajectory per instance over 1,000 resamples. The controlled experiments enumerate their complete constructed test sets, so confidence intervals are reserved for the sampled natural-trace evaluation. An issue-text-only contract condition appears in Appendix B.

Across the evaluation, the main denominators come from complementary tests rather than one pooled benchmark: 180 controlled fault and matched-control transactions, 56 acquired-workflow variants, a broader 280-workflow accept/reject study, 84 natural-trajectory condition evaluations, a 504-context actor-mistake study, 320 language perturbations, and 500 forced or benign concurrency trials. Semantic-refactor, emulator, and resampling studies test additional boundaries. We report each denominator separately because these studies answer different questions and should not be interpreted as independent samples from a single deployment distribution.

4 Results

4.1 State explains most of the composition gain

Figure 2a shows that no single feature explains the full result. Transaction history catches every continuity fault and portions of the scope and rollback classes, reaching 71/90. Cumulative-scope tracking catches the remaining scope faults and three more rollback faults. The restoration oracle catches three rollback functions that run but do not restore the expected state.

The final five faults modify the expected paths but restore the wrong content, such as an intermediate rather than initial file state. Checking the restored state against the original predicate catches these cases. This layer contributes the final 5/90 detections, or 5.6 percentage points over the strongest matched baseline. The one-solver and two-solver configurations make identical decisions; the second solver is a cross-check, not the source of the accuracy gain. These are consequential failures despite their count: an executor that records rollback as successful can otherwise allow later actions to proceed from an intermediate workspace state.

4.2 The same rollback gap appears on acquired workflows

Figure 2b shows the same separation on acquired workflows. All methods detect missing required fields, so field-drop performance alone does not distinguish ACTIONPROOF. Cross-step faults do. The stateful baseline detects 19/24, while ACTIONPROOF detects 24/24. The five disagreements

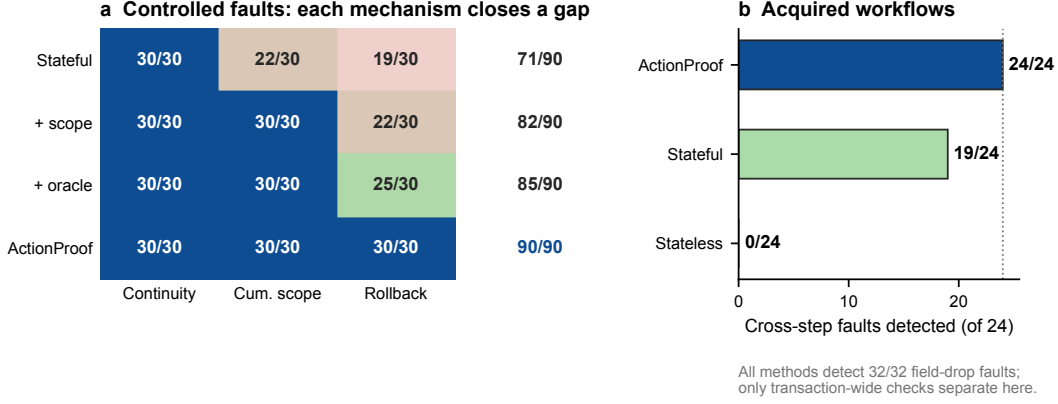


Figure 2: **Mechanism and transfer.** (a) Detected controlled cross-step faults as state, cumulative scope, restoration, and predicate-level rollback checks are added. All methods accept 90/90 matched valid controls. (b) Cross-step detections on eight acquired workflows; all three methods also detect 32/32 missing-field variants.

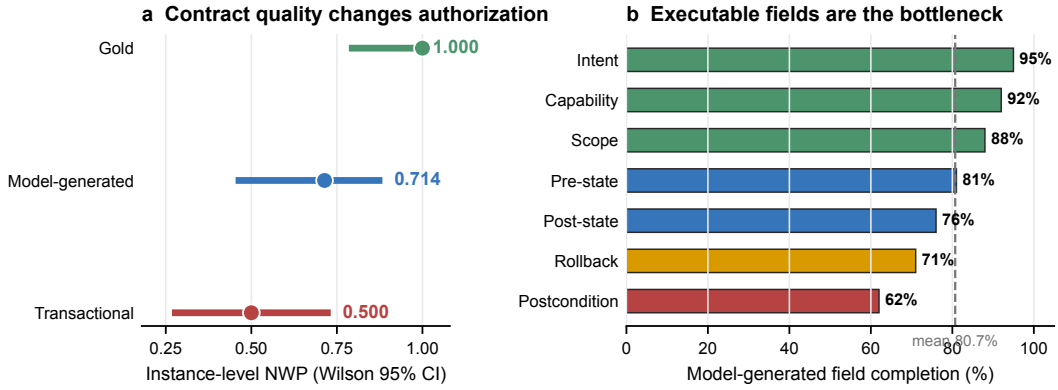


Figure 3: **Contract quality and its bottleneck.** (a) Instance-level NWP on 14 distinct SWE-bench instances; intervals are Wilson 95% CIs. (b) Completion of model-generated contract fields. Postconditions and rollback descriptions are the weakest executable fields.

are predicate-level rollback faults, matching the controlled-frame failure class. Across the broader repository run, all 70 complete workflows are authorized and all 210 incomplete or reordered variants are blocked.

The strongest stateful baseline also misses two of six out-of-order emulator variants in Section 6.4. Across the controlled, acquired, and emulator frames, this gives 12 cases in which state history alone does not establish the required final or restored state. We keep the three denominators separate rather than treating them as one statistical sample.

4.3 Model-generated contracts improve natural-trace authorization

Trajectory-derived reference contracts preserve every needed write without extra changes. Model-generated contracts preserve needed writes on 10/14 distinct instances, compared with 7/14 for transaction-level authorization. At trajectory granularity, they reduce extraneous writes from 11 to 3 while preserving NWP on 15/21 trajectories. The six remaining trajectory-level errors split evenly between overly narrow scope and weak executable fields. This points to specific acquisition improvements rather than treating contract quality as a single opaque problem.

The transactional baseline is intentionally permissive: it requires no generated contract and authorizes the complete observed trajectory write set. Its lower NWP follows from the absence of a mechanism for separating needed from extraneous writes, not from a stricter specification burden. Conversely, the

model-generated condition must infer a contract from repository context and then satisfy per-write checks. We use the transactional baseline as a specification-free comparison, not as an upper bound on contract-aware methods.

Sampling one trajectory per distinct instance preserves the method ordering; median NWP estimates remain within 0.02 of the instance-level point estimates. The analysis therefore supports a contract-quality effect at the evaluated scale. Appendix C reports the corresponding uncertainty and separate natural-fault denominator.

5 Acquiring Useful Contracts

Figure 3b helps explain the natural-trace result. Intent, capability, and scope can often be recovered from repository context. Model-generated contracts complete 80.7% of fields on average, including 62% of postconditions and 71% of rollback descriptions. These contracts use one zero-temperature response per distinct instance, without reranking. The observed 10/14 NWP therefore describes the operating point of this single-shot contract producer rather than a ceiling for the checker.

The relationship between field completion and NWP is nonlinear. High intent and capability coverage cannot compensate for a missing postcondition that allows an unnecessary but in-scope write, or for a rollback description that cannot restore observed state. On the 14-instance evaluation, removing one executable postcondition lowers NWP from 1.000 to 0.857. The most useful next research target is therefore not generic contract generation, but synthesis and validation of executable postconditions and rollback specifications.

6 Additional Checks

6.1 Semantic postconditions

Byte equality rejects behaviorally equivalent code. Across eight repositories, a byte-hash oracle accepts 0/8 behavior-preserving refactors, while a repository test oracle accepts 8/8 and ACTION-PROOF accepts the same 8/8 through its postcondition field. The refactors include variable renaming, import reordering, whitespace normalization, docstring and comment updates, type annotations, an equivalent list comprehension, and an equivalent f-string conversion.

This experiment tests the choice of postcondition, not proof carrying by itself. Any checker using the same repository test could make the same decision.

6.2 Natural-language invariance

Request prose is not part of the SMT query. Across 320 synonym, passive-voice, filler, and prompt-injection perturbations, the decision changes zero times. This experiment checks that the implementation follows the intended typed interface.

6.3 Commit-time state drift

A final state check closes the window between authorization and commit. Without it, the executor commits on stale state in all 200 forced races. With it, all 200 races are stopped before commit. The same check catches all 50 multi-file races. Appendix C reports its behavior under benign concurrent edits.

This follows the validation phase of optimistic concurrency control [21]; the result is an implementation test, not a production-deployment claim.

6.4 Emulated apply-time composition

Three ordered infrastructure workflows execute successfully against a local AWS emulator under both ACTIONPROOF and the stateful transactional baseline. Across six out-of-order variants, ACTIONPROOF blocks 6/6 and the stateful baseline blocks 4/6. The two missed cases have locally valid steps but violate the signed cross-step order. Appendix C describes what the emulator does and does not cover.

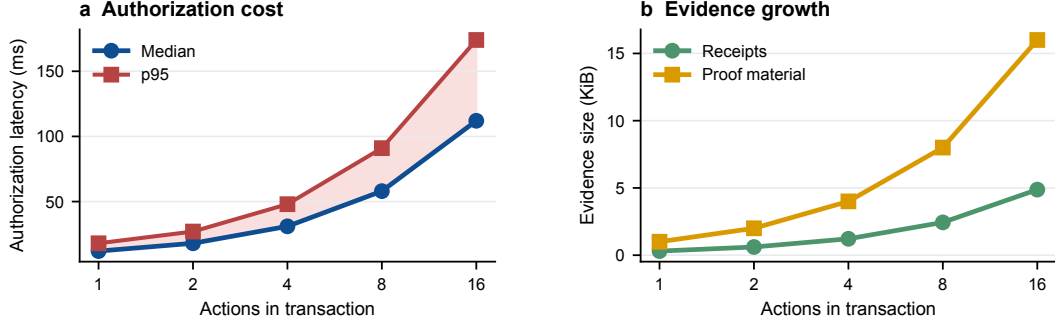


Figure 4: Authorization latency and evidence size by transaction length. Both grow approximately linearly over the measured range from one to 16 actions.

6.5 Latency and artifact growth

At four actions, median authorization time is 31 ms and p95 is 48 ms. The 16-action median is 112 ms. Receipt and proof material grow linearly over the measured range; a component-level timing breakdown is included in the supplement.

7 Related Work

Coding agents and repository benchmarks. SWE-bench and SWE-agent established executable, issue-level repository repair [1, 2]. OpenHands develops a general software-agent platform [3], while SWE-Lancer, SWE-PolyBench, and automated benchmark generation broaden the task and repository distributions used for evaluation [4–6]. These works ask whether an agent completes a software task. ACTIONPROOF asks a complementary runtime question: whether its state-changing sequence satisfies one transaction contract before durable commit.

Runtime enforcement for agents. Recent security benchmarks expose failures around tools, memory, and prompt injection [7, 8]. CaMeL separates trusted control flow from untrusted data and enforces capabilities at tool calls [9]. AgentSpec and Progent express runtime constraints and privileges [10, 11], while FORGE supports history-sensitive policies through an external reference monitor [12]. ActPlane covers execution paths at the operating-system layer [13], and VIGIL checks temporal and value-flow policies over finite agent traces using SMT [14]. ACTIONPROOF shares their use of enforcement outside the model, but specializes the checked object to a filesystem transaction with cumulative write scope, executable postconditions, receipt-bound observations, commit-time rechecks, and verified rollback.

Proof-bearing agent governance. Recent proposals attach portable certificates to agent actions [15], require certification before trace execution [16], authenticate execution traces independently of the host [17], or derive temporary execution authority from proof artifacts [18]. ACTIONPROOF is narrower and empirical: it defines a concrete local action language, executes and rolls back file mutations, and measures which cross-step faults transaction-wide checking catches on controlled and acquired coding workflows.

Contract construction. Autoformalization work translates natural-language agent instructions and tool descriptions into policy-as-code [19]. ACTIONPROOF does not claim to solve contract synthesis. Instead, the gap between model-generated and gold contracts measures how enforcement quality depends on the executable contract presented to the checker.

Foundations and implementation. Proof-carrying code supplies the producer/checker separation [20]; the final state recheck follows optimistic concurrency control [21]. The prototype realizes policy and satisfiability checks with OPA, Z3, and cvc5 [22–24].

8 Limitations

ACTIONPROOF cannot repair a contract that leaves out the user’s real intent. Its guarantee also depends on the trusted components enumerated in Section 2: the policy and contract interpretation, action interpreter, snapshots, execution monitor, solver encoding, and rollback implementation. Dual solvers reduce dependence on one solver implementation but not on the shared encoding. The controlled faults test properties built into the transaction model; they do not estimate real-world fault frequency.

The natural-trace result measures needed-write preservation rather than SWE-bench patch correctness. It covers 14 distinct instances across seven repositories, so the confidence intervals remain important. The action language is limited to local disposable workflows and an emulator. Full assumptions and sensitivity analyses appear in Appendix C.

9 Conclusion

ACTIONPROOF checks coding-agent transactions as transactions, rather than as unrelated tool calls. This matters most when a failure appears only across steps. It detects all 90 controlled cross-step faults and all 24 acquired cross-step faults, including five rollback errors missed by the strongest matched stateful baseline.

On natural traces, model-generated contracts reach 0.714 needed-write preservation, compared with 0.500 for transaction-level authorization and 1.000 for trajectory-derived reference contracts. The checker is effective once the contract captures the right behavior. The next challenge is generating executable postconditions and rollback descriptions reliably from repository context.

Artifact Availability

The companion artifact contains contracts, traces, solver outputs, analysis scripts, and instructions for rebuilding the paper’s tables and figures. Running `./reproduce.sh deterministic` rebuilds every reported table from stored outputs in approximately three minutes without API keys or network access.

References

- [1] C. E. Jimenez et al. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? *ICLR*, 2024.
- [2] J. Yang et al. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. *NeurIPS*, 2024.
- [3] X. Wang et al. OpenHands: An Open Platform for AI Software Developers as Generalist Agents. *ICLR*, 2025.
- [4] S. Miserendino, M. Wang, T. Patwardhan, and J. Heidecke. SWE-Lancer: Can Frontier LLMs Earn \$1 Million from Real-World Freelance Software Engineering? *arXiv:2502.12115*, 2025.
- [5] M. S. Rashid et al. SWE-PolyBench: A Multi-Language Benchmark for Repository-Level Evaluation of Coding Agents. *arXiv:2504.08703*, 2025.
- [6] K. Vergopoulos, M. N. Müller, and M. Vechev. Automated Benchmark Generation for Repository-Level Coding Tasks. *DL4C and FMWild at ICLR*, 2025.
- [7] H. Zhang et al. Agent Security Bench (ASB): Formalizing and Benchmarking Attacks and Defenses in LLM-Based Agents. *ICLR*, 2025.
- [8] I. Evtimov, A. Zharmagambetov, A. Grattafiori, C. Guo, and K. Chaudhuri. WASP: Benchmarking Web Agent Security Against Prompt Injection Attacks. *arXiv:2504.18575*, 2025.
- [9] E. Debenedetti et al. Defeating Prompt Injections by Design. *arXiv:2503.18813*, 2025.

- [10] H. Wang, C. M. Poskitt, and J. Sun. AgentSpec: Customizable Runtime Enforcement for Safe and Reliable LLM Agents. *ICSE*, 2026.
- [11] T. Shi et al. Progent: Securing AI Agents with Privilege Control. arXiv:2504.11703, 2025.
- [12] N. Palumbo, S. Choudhary, J. Choi, G. Amir, P. Chalasani, and S. Jha. Formal Policy Enforcement for Real-World Agentic Systems. arXiv:2602.16708, 2026.
- [13] Y. Zheng et al. ActPlane: Programmable OS-Level Policy Enforcement for Agent Harnesses. arXiv:2606.25189, 2026.
- [14] Y. Li et al. VIGIL: Runtime Enforcement of Behavioral Specifications in AI Agent Skills. arXiv:2606.26524, 2026.
- [15] Z. Wang. Proof-Carrying Agent Actions: Model-Agnostic Runtime Governance for Heterogeneous Agent Systems. arXiv:2606.04104, 2026.
- [16] X.-Y. Liu Yanglet, X. Wang, and A. Capponi. No Certificate, No Execution: Certified Traces as a Foundation for Trustworthy AI Agents. arXiv:2605.24462, 2026.
- [17] A. Grigor, C. Schroeder de Witt, S. Birnbach, and I. Martinovic. VET Your Agent: Towards Host-Independent Autonomy via Verifiable Execution Traces. arXiv:2512.15892, 2025.
- [18] J. He and D. Yu. Verifiable Agentic Infrastructure: Proof-Derived Authorization for Sovereign AI Systems. arXiv:2605.15228, 2026.
- [19] A. Mondl, M. Maisel, and J. H. Brock. Autoformalization of Agent Instructions into Policy-as-Code. *AIWILD at ICML*, 2026.
- [20] G. C. Necula. Proof-Carrying Code. *POPL*, 1997.
- [21] H. T. Kung and J. T. Robinson. On Optimistic Methods for Concurrency Control. *ACM Transactions on Database Systems*, 6(2):213–226, 1981.
- [22] Open Policy Agent Project. OPA and Rego Documentation. 2026.
- [23] L. de Moura and N. Bjørner. Z3: An Efficient SMT Solver. *TACAS*, 2008.
- [24] H. Barbosa et al. cvc5: A Versatile and Industrial-Strength SMT Solver. *TACAS*, 2022.

A Complete proof of the composition guarantee

Proof of Theorem 1. We prove Items 1–4 by induction on the length n of an accepted prefix. For $n = 0$, the prefix is empty, $W_0 = \emptyset$, and the claims hold vacuously.

Assume the claims hold after $n - 1$ committed actions. Before action n , the fail-closed monitor evaluates $\phi_n(x_{n-1})$, verifies $S_n \subseteq P.\text{allowed_scope}$, verifies $W_{n-1} \cup S_n \subseteq P.\text{transaction_budget}$, checks the required capabilities and signed digests, and requires both solvers to reject the bounded violation formula. A failed check prevents execution or commit. Therefore an accepted step satisfies Item 1. By the induction hypothesis, $W_{n-1} = \bigcup_{i=1}^{n-1} S_i$; the cumulative-scope check and ledger update give $W_n = \bigcup_{i=1}^n S_i \subseteq P.\text{transaction_budget}$, proving Item 2.

After execution, commit occurs only when $\psi_n(x_n) = 1$ and $\tau_n(x_n) = 0$, so Item 3 is preserved. The update

$$h_n = H(h_{n-1} \parallel \text{canonical}(c_n) \parallel \text{snapshot}(x_n))$$

binds the prior head, canonical contract, and observed post-state. Collision resistance and signature verification make any undetected change to one of these values an explicit violation of the theorem assumptions. This proves Item 4.

For Item 5, suppose the first failure occurs after actions $1, \dots, k - 1$ commit. By assumption, each rollback description ρ_j is executable and maps the post-state of action j to the signed pre-state of that action. The monitor applies $\rho_{k-1}, \dots, \rho_1$ in reverse order and checks the corresponding pre-state predicate after every application. Induction over this reverse sequence yields $x_{k-2}, x_{k-3}, \dots, x_0$. Thus the final observed workspace is x_0 . If any description is incorrect or non-executable, the antecedent of Item 5 is false and the implementation enters INCOMPLETE-ROLLBACK, as specified. $\square$

B Issue-text retrieval sensitivity

We also build contracts using only file paths and state cues found in issue text. This is a deliberately narrow source of information, so we report it separately from the repository-context model used in the main comparison.

Condition	Instance NWP	95% CI	Trajectory NWP	Extra writes
Trajectory-derived reference	14/14 (1.000)	[0.785, 1.000]	21/21	0
Model-generated	10/14 (0.714)	[0.454, 0.883]	15/21	3
Transactional	7/14 (0.500)	[0.268, 0.732]	10/21	11
Issue-text retrieval	3/14 (0.214)	[0.076, 0.476]	5/21	0

Table 1: Full contract-source sensitivity matrix. Instance-level outcomes are primary; trajectory-level counts are included for completeness.

The issue-text condition blocks 16/21 trajectories: nine contracts omit a needed path, four omit rollback descriptions, and three contain an incorrect pre-state predicate. Its mean field completion is 30.4%, compared with 80.7% for the repository-context model. Issue prose alone is therefore a poor source for stateful executable contracts.

C Statistical and implementation boundaries

Endpoint. All 21 natural trajectories are recorded as submitted; the SWE-bench FAIL_TO_PASS oracle was not recovered. NWP therefore measures authorization conformance over needed and extraneous writes, not patch correctness. The reference contract is derived from the submitted file set.

Cluster sensitivity. Seven trajectories repeat one high-activity instance under different agent configurations. Sampling one trajectory per instance over 1,000 resamples preserves the ordering of the reported methods; median NWP estimates remain within 0.02 of the instance-level estimates.

Natural and controlled faults. In a separate 504-context actor-mistake study, 16 contexts contain observed actor errors and 427 contain controlled injected faults. ACTIONPROOF misses none in either category. Wilson 95% upper bounds are 19.4% for the natural slice (0/16) and 0.89% for the controlled slice (0/427). We keep these denominators separate because the natural and constructed samples answer different questions.

Commit-time recheck. The rechecked executor detects 50/50 multi-file races. In 50 benign concurrent-edit trials, conservative directory-level hashing produces three false aborts when an out-of-scope file in the same directory changes. Finer snapshot granularity is a direct implementation improvement.

Environment scope. The apply-time workflows run against a local AWS emulator. They do not model IAM latency, eventual consistency, cross-region propagation, or production deployment. The measured latency results cover chain lengths 1 through 16 and support linear growth only over that range.

D Experimental and Reproducibility Details

The controlled evaluation contains 30 four-action workflows, three fault types per workflow, and one valid control matched to each faulted transaction. The acquired extension contains eight workflows with 32 field-drop and 24 cross-step variants. The natural analysis uses 21 trajectories from 14 distinct SWE-bench Verified instances across seven repositories.

D.1 Model use

Claude Sonnet 5 (claude-sonnet-5-20250514) generated one candidate contract for each distinct natural-trace instance from issue descriptions, repository file trees, and relevant source files. The 14

calls were made from August 4 to August 6, 2026, with temperature 0, top_p=1, and a 4,096-token output limit. The first response was used without retries or reranking. Repeated trajectories for the same instance share one contract. The model does not judge its own contracts or score any method; contract checking, baseline execution, and metric computation are deterministic.

D.2 Compute

All experiments ran on one AWS EC2 m5.2xlarge worker with eight vCPUs, 32 GiB RAM, a 200 GiB EBS volume, and Ubuntu 22.04. No GPU was used. The software environment used OPA 0.68.0, Z3 4.13.0, and cvc5 1.1.2. Controlled experiments completed in approximately 14 minutes, acquired-workflow experiments in 8 minutes, natural-trace experiments in 22 minutes, and the supporting experiments in 35 minutes. Including preliminary and debugging runs, the project used approximately 10 instance-hours.

D.3 Reproduction

The anonymous supplement contains contracts, policies, trajectory records, model responses, baseline decisions, solver inputs and outputs, execution and rollback traces, table-generation scripts, documentation, and licenses. `./reproduce.sh deterministic` rebuilds all paper tables from stored outputs without network access or API credentials. It requires Python 3.11 and the dependencies in `environment.yml` and completes in approximately three minutes. The optional `./reproduce.sh full` path reruns the solvers and contract-generation calls, requires model API credentials, and completes in approximately 80 minutes. Because model responses can vary, the stored responses remain the basis of the reported results.

D.4 External assets

Asset	Version	License	Distribution
SWE-bench Verified	2.1.0	MIT	Metadata referenced; source not redistributed
SWE-agent	0.7.0	MIT	Trajectory logs included
OPA	0.68.0	Apache 2.0	Referenced; binary not included
Z3	4.13.0	MIT	Referenced; binary not included
cvc5	1.1.2	BSD 3-Clause	Referenced; binary not included
LocalStack	3.7.2	Apache 2.0	Referenced; image not included
Repository snapshots	Instance-specific	Repository license	Checked out during reproduction

Table 2: Versions and redistribution status for external assets. The supplement includes license notices and does not redistribute repository source snapshots.

E Broader impact and release scope

ACTIONPROOF can make agent changes easier to inspect and safer to delegate. Its main risk is misplaced confidence in an incomplete contract; an overly narrow contract can also block useful work. A practical deployment should show the active contract to an operator, retain denied and rollback traces, and stop for review when rollback cannot restore the expected state. The released materials contain bounded local authorization traces rather than an offensive model or tool.